



**Universidad
Tecnológica
del Perú**

**“Aplicación de Design Thinking para la mejora de los trámites
municipales”**

Integrantes:

Beraun García, Leonardo Ángel – 70%

Macedo Salas, Edhu Anthony – 100%

Matos Chuquino, Jesús Presencio – 100%

Asistente Virtual:

Manuelito

Curso:

Diseño de Productos y Servicios

Sección:

24896

2026 – I

1. Prototipado rápido y Pruebas de Concepto

1.1. Historia de Usuario Validada (HU)

Se ha seleccionado la HU de mayor valor para el ciudadano, la cual integra el requerimiento de la Inteligencia Artificial desde la fase inicial (MVP):

- HU-02
- Título: Visualización detallada del trámite con Asistente de Comprensión IA.
- Descripción: Como ciudadano peruano, quiero visualizar de forma clara y 100% pública el detalle de un trámite municipal (costos, tiempos, requisitos), incluyendo un resumen en lenguaje sencillo (generado por IA) y un chatbot interactivo, para comprender el proceso burocrático sin necesidad de contratar a un tramitador.
- Justificación de Negocio: Reduce la fricción cognitiva y la brecha digital.

1.2. Criterios Given–When–Then (Criterios de Aceptación)

Escenario 1: Interacción con el Asistente Virtual (Chatbot IA)

Given Dado que el ciudadano se encuentra en la pantalla de detalle del trámite "Licencia de Funcionamiento" y tiene dudas sobre un requisito técnico. When Cuando el usuario escribe la pregunta *"¿El croquis se puede hacer a mano?"* en el panel lateral del Chatbot y presiona enviar. Then Entonces el sistema procesa la consulta y el Asistente IA responde de manera contextual (ej. *"Sí, puedes hacerlo a mano alzada marcando las calles principales"*), simulando un tiempo de procesamiento humano.

Escenario 2: Visualización de Datos Oficiales

Given: Dado que el usuario seleccionó un trámite desde el Catálogo Principal.
When: Cuando carga la vista de detalle. Then: Entonces la pantalla debe dividirse mostrando un 70% para la información oficial y un 30% fijo para el Asistente IA.

1.3. Prototipo wireframe tipo Balsamiq (Estructura UI/UX)

El diseño respeta el principio de "Carga Cognitiva Reducida", dividiendo la interfaz en dos bloques lógicos:

Volver al Catálogo

MuniVirtual

Trámite: Licencia de Funcionamiento Comercial

Costo: S/ 150 días hábiles: 5 días tipo: Virtual

★

Resumen Inteligente IA: Este trámite te permite abrir locales menores a 100m2. Solo necesitas tu DNI, contrato de alquiler y un extintor vigente.

Requisitos Oficiales:

- ☐ DNI vigente
- ☐ Declaración Jurada
- ☐ Croquis de Distribución

★ Asistente de IA del Trámite

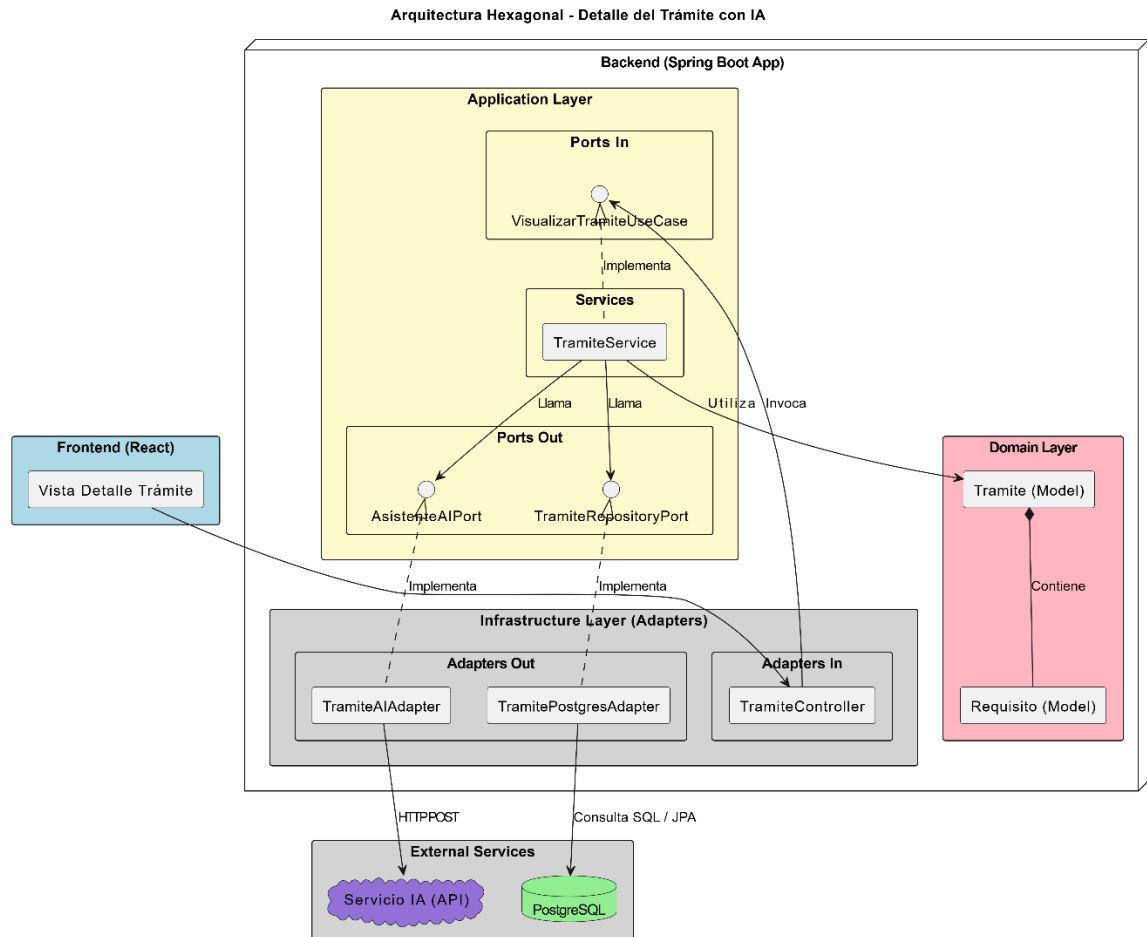
IA: El croquis de distribución puede ser dibujado a mano alzada, no necesitas un arquitecto para locales pequeños.

Pregúntale a la IA sobre este trámite... Ej: ¿Cómo hago el croquis?

Enviar

1.4. Diagrama de Arquitectura

El backend está estrictamente basado en Arquitectura Hexagonal (Puertos y Adaptadores) para asegurar la independencia de la tecnología y el uso temporal de la IA simulada.



1.5. Estructura del Repositorio GitHub

La organización del proyecto separa claramente las capas y las responsabilidades:

/muniGo

```
|— database/
|  |— init.sql          # Script SQL (Tablas y Trámites Semilla)
|— backend/            # Spring Boot (Java)
|  |— src/main/java/com/muni/tramites/
|    |— domain/        # Entidades core puras
|    |— application/   # Interfaces (Ports) y Casos de Uso
|    |— infrastructure/ # Controllers, Repositories (JPA) y Adaptador IA
|— frontend/           # React + Vite
|  |— src/
|    |— components/
|      |— CatalogoTramites.jsx # Vista H1
|      |— DetalleTramite.jsx  # Vista H2
|      |— PanelChatbot.jsx    # Componente IA Simulado
|    |— App.jsx            # Enrutador principal
|— tailwind.config.js
|— package.json
```

1.6. Selección Tecnológica Justificada

- **Frontend (React + Tailwind CSS):** Se eligió React por su modelo de componentes, lo que permite crear el Panel de Chatbot de forma modular y mantener su estado (historial de mensajes) sin recargar la página. Tailwind CSS permite diseñar interfaces modernas rápidamente, vital para el "Design Thinking".
- **Backend (Java + Spring Boot):** Es el estándar empresarial por excelencia. Facilita la implementación rigurosa de la Arquitectura Hexagonal y la Inversión de Dependencias (IoC).
- **Base de Datos (PostgreSQL):** Un motor relacional robusto. Se configuró localmente (spring.jpa.show-sql=true) para auditar la inserción y lectura de requisitos legales garantizando integridad.
- **IA Simulado (Mock Adapter en Backend/Frontend):** Para efectos de este MVP, la IA se inyecta mediante *Mocks* usando demoras asíncronas (setTimeout) y detección de palabras clave, ahorrando costos de API pero validando la Arquitectura.

1.7. Código Funcional Inicial

A continuación, se muestra el fragmento clave del componente React que gestiona el chatbot inteligente simulado:

```
useEffect(() => {
  if (tramite) {
    setMessages([
      {
        id: 1,
        sender: 'ia',
        text: `¡Hola! Soy tu Asistente Municipal IA. Estoy aquí para ayudarte específicamente con tu consulta sobre "${tramite.nombre}". ¿Qué duda tienes sobre los requisitos o el proceso?`
      }
    ]);
  }
}, [tramite]);

useEffect(() => {
  messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
}, [messagesEndRef]);
```

```

    }, [messages, isTyping]));

const handleSubmit = (e) => {
  e.preventDefault();
  if (!input.trim()) return;

  const userMessageText = input;
  const userMessage = {
    id: Date.now(),
    sender: 'user',
    text: userMessageText
  };

  setMessages((prev) => [...prev, userMessage]);
  setInput('');
  setIsTyping(true);

  setTimeout(() => {
    let replyText = '';
    const normalizedQuery = userMessageText.toLowerCase();

    if (normalizedQuery.includes('croquis') ||
normalizedQuery.includes('dibujo')) {
      replyText = 'El croquis de distribución puede ser dibujado a mano
alzada, no necesitas un arquitecto para locales pequeños. Solo marca calles
principales e ingresos.';
    } else if (normalizedQuery.includes('fut') ||
normalizedQuery.includes('formulario')) {
      replyText = `El Formulario Único de Trámite (FUT) es totalmente
gratuito y se pide en mesa de partes para iniciar la solicitud de
"${tramite.nombre}";`;
    } else if (normalizedQuery.includes('tiempo')) {
      replyText = `Este trámite (${tramite.nombre}) demora aproximadamente
${tramite.tiempoEstimado} hábiles una vez entregados todos los requisitos
oficiales.`;
    } else {
      replyText = `Para resolver dudas adicionales sobre el trámite de
"${tramite.nombre}", te invitamos a acercarte a la ventanilla de atención
al ciudadano en el palacio municipal.`;
    }
  })
}

```

1.8. Enlaces de Entrega

- Enlace al Repositorio GitHub: <https://github.com/Estudiante-leonardo/Muni-Go.git>
- Video Demostrativo del Funcionamiento: [Video-Productos.mp4 - Google Drive](#)

2. Iteración y mejora continua

2.1. Explicación conceptual del Ciclo Deming (PDCA)

El ciclo Deming o ciclo Plan-Do-Check-Act (PDCA) es una metodología de mejora continua aplicada para optimizar procesos, productos y servicios. En el contexto de nuestro proyecto Muni-Go, lo aplicamos de la siguiente manera:

- **PLAN (Planificar):**

Identificar la fricción que sufren los ciudadanos al buscar trámites (ej. no saber los pasos a seguir o dónde descargar formularios) y planificar soluciones de UI/UX y arquitectura backend para resolverlo.

- **DO (Hacer):**

Desarrollar e integrar las mejoras en el código. Esto incluye la creación de nuevos componentes en React (cajas de descarga, líneas de tiempo, selectores de distrito) y la adaptación de los controladores y servicios en Spring Boot.

- **CHECK (Verificar):**

Validar si las nuevas funcionalidades en la interfaz realmente reducen la carga cognitiva del usuario y comprobar si el Asistente IA (simulado) responde adecuadamente sin romper el flujo de la aplicación.

- **ACT (Actuar):**

Estandarizar los componentes de UI exitosos para reutilizarlos en otros trámites (ej. un componente universal de "Pasos") y consolidar la arquitectura hexagonal del backend para que la futura integración con una API de IA real sea limpia y sin fricciones.

2.2. Objetivo de la práctica

Aplicar de forma práctica el ciclo PDCA para estructurar y documentar la mejora continua de la plataforma "Muni-Go" (Multi-distrito).

El objetivo específico de esta iteración trasciende la simple codificación: buscamos perfeccionar la Experiencia de Usuario (UX) de la vista de catálogo y detalle de trámites, evolucionando de un prototipo informativo básico a una interfaz altamente interactiva y accionable (que incluya geolocalización por distrito, descarga de formatos FUT y flujos secuenciales). Asimismo, se busca validar el comportamiento y la utilidad de un Asistente de IA (actualmente en fase de simulación) para asegurar que la arquitectura tecnológica (React + Spring Boot) esté lista, testeada y modularizada para la inyección del modelo de Inteligencia Artificial real en los próximos sprints.

2.3. Caso de estudio

La aplicación desarrollada (Muni-Go) permite:

- Seleccionar y cambiar la municipalidad de interés en tiempo real.
- Visualizar información detallada de trámites (costos, tiempos, requisitos).
- Descargar formatos oficiales (FUT) y visualizar los pasos a seguir de forma secuencial.
- Ubicar la sede física correspondiente al trámite seleccionado.
- Interactuar con un Asistente IA (simulado en esta iteración mediante retardos asíncronos) para resolver dudas de manera conversacional y en lenguaje sencillo.

2.4. Requisitos técnicos de la aplicación

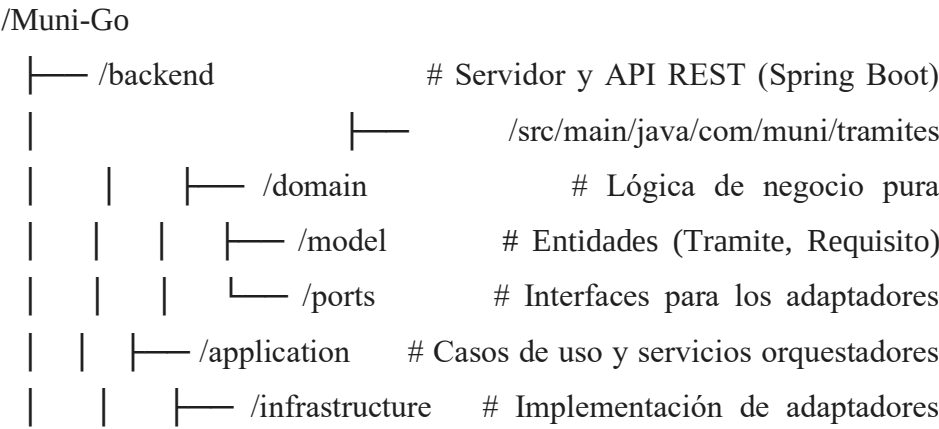
La solución implementa una Arquitectura Hexagonal en el backend (Spring Boot) y un frontend modular (React), dividiéndose en:

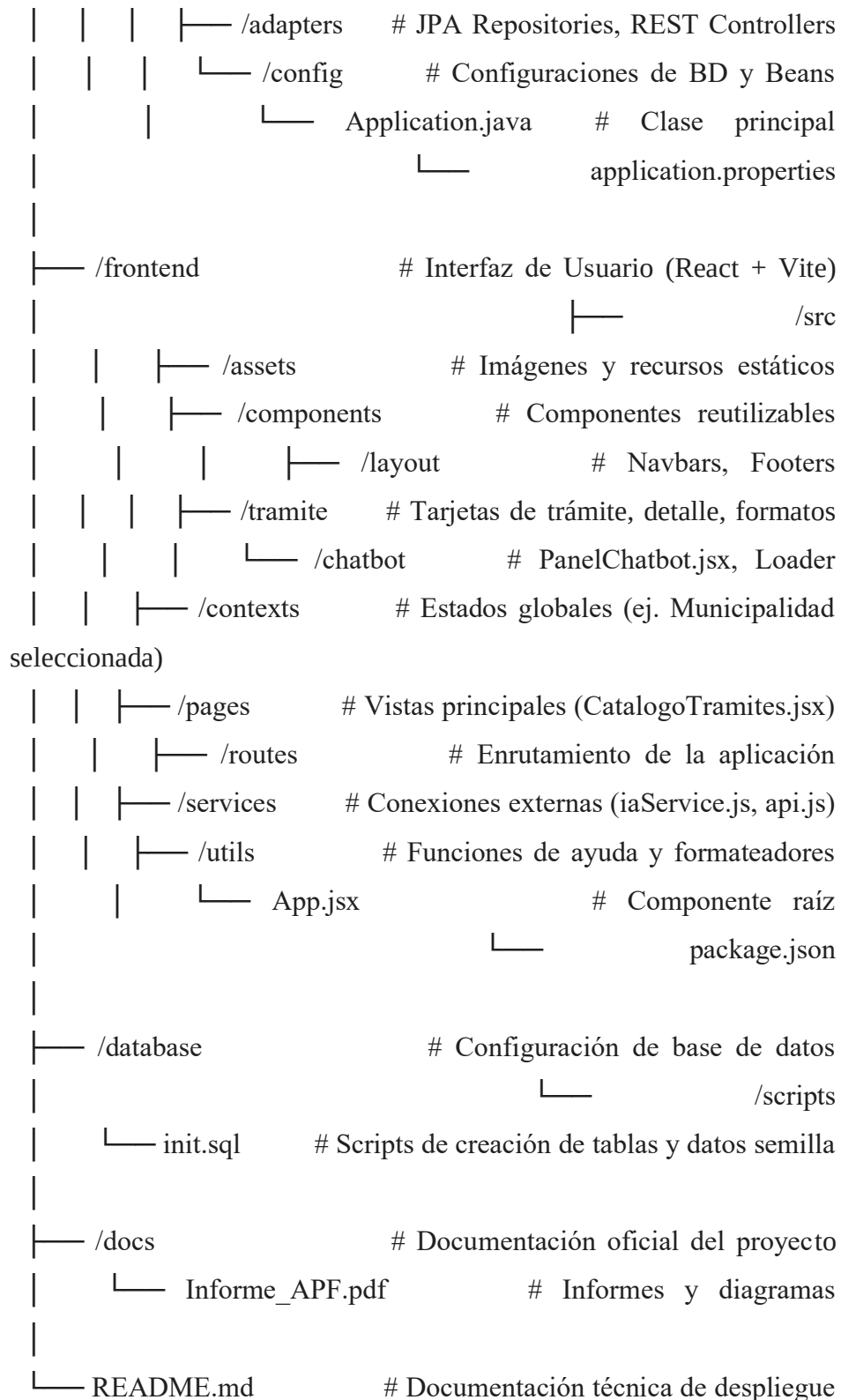
Capa	Responsabilidad
------	-----------------

Vista (Frontend)	Interfaces de usuario en React (Catálogo, Detalle, Panel Chatbot).
Controlador / Adapters In	Recepción de solicitudes REST en Spring Boot.
Modelo / Domain	Lógica de negocio pura (Entidades: Trámite, Requisito, Municipalidad).
Base de Datos / Adapters Out	Persistencia de datos mediante PostgreSQL y JPA.
Integración IA (Simulada)	Servicios en el frontend/backend que emulan respuestas de IA mediante retardos asíncronos y análisis de palabras clave.

2.5. Estructura GitHub esperado

Para garantizar el cumplimiento de la arquitectura hexagonal y la modularidad del frontend, la organización del repositorio Muni-Go es la siguiente:





2.6. Historia de Usuario Principal

Historia de Usuario (HU-03: Visualización Integral y Asistencia)

- **Como** ciudadano peruano.
- **Necesito** consultar los requisitos, formatos descargables, pasos a seguir y ubicación física de un trámite, además de poder seleccionar mi municipalidad específica.
- **Para** prepararme correctamente antes de ir presencialmente y resolver mis dudas inmediatas mediante un asistente de IA (simulado).

2.7. Criterios de aceptación

- **Escenario 1 - Cambio de Municipalidad dinámico:**
 - *Given* que el usuario se encuentra en la vista de detalle de un trámite.
 - *When* selecciona una nueva municipalidad en el botón superior derecho.
 - *Then* el sistema actualiza la vista mostrando la dirección ("¿Dónde ir?") y los costos/requisitos específicos de ese distrito.
- **Escenario 2 - Descarga de Formatos y Pasos:**
 - *Given* que el ciudadano revisa los requisitos de la Licencia de Funcionamiento.
 - *When* se desplaza hacia abajo en la pantalla principal.
 - *Then* el sistema muestra una sección clara con el "Formato Único de Trámite" habilitado para descarga y una línea de tiempo con los pasos 1, 2 y 3 a seguir.
- **Escenario 3 - Interacción con IA Simulada:**
 - *Given* que el paciente tiene dudas sobre cómo hacer un croquis.
 - *When* escribe la pregunta en el panel lateral derecho del Asistente y presiona enviar.
 - *Then* el sistema muestra un indicador de "escribiendo..." y luego de unos segundos, devuelve una respuesta contextual simulada resolviendo la duda.

2.8. Wireframe simplificado

Área	Componentes
------	-------------

Encabezado	Botón de retorno, Nombre del sistema, Botón Selector de Municipalidad.
Cabecera del Trámite	Título, Costo, Días Hábiles, Tipo de trámite, Resumen Inteligente IA.
Cuerpo Principal	Checkbox de Requisitos Oficiales.
Nuevas Secciones (Mejora)	Caja de "Formatos para descargar" (FUT), Línea de tiempo "Pasos a seguir", Caja de ubicación "¿Dónde ir?".
Panel Lateral (Derecho)	Asistente de IA del Trámite (Chatbot) con historial de mensajes e input de texto.

A Web Page
https://MuniGo/Descripcion_Catalogo

Volver al Catálogo

MuniVirtual

Trámite: Licencia de Funcionamiento Comercial

Costo: S/ 150 dias habiles: 5 dias tipo: Virtual

★ **Resumen Inteligente IA:** Este trámite te permite abrir locales menores a 100m2. Solo necesitas tu DNI, contrato de alquiler y un extintor vigente.

★ **Asistente de IA del Trámite**

IA: El croquis de distribución puede ser dibujado a mano alzada, no necesitas un arquitecto para locales pequeños.

Pregúntale a la IA sobre este trámite... Ej: ¿Cómo hago el croquis?

Enviar

Requisitos Oficiales:

☐ DNI vigente
☐ Declaración Jurada
☐ Croquis de Distribución

Formatos para descargar

Imprime y llena estos documentos desde casa para evitar colas o buscar copias el mismo día.

Formato Único de Trámite (FUT)
Documento PDF - 120 KB

Pasos a seguir:

1

Preparar Expediente
Junta todos los requisitos y llena los formatos descargados en un folder.

2

Acercarse a Sede
Dirígete a la Municipalidad de Carabayllo. Si tu trámite tiene costo (S/ 120), pasa primero por caja.

3

Mesa de Partes
Entrega tus documentos. Te asignarán un número para que consultes luego el resultado.

¿Dónde ir?

Municipalidad de Carabayllo

Plaza Central del distrito
Horario: L-V de 8:00 AM a 4:30 PM

2.9. Organización de actividades (90 minutos)

Actividad	Tiempo
Explicación conceptual PDCA	10 min
Revisión de aplicación y GitHub	15 min

Desarrollo de cuadro PDCA	40 min
Presentación de mejoras	15 min
Retroalimentación	10 min

2.10. Cuadro Integrado de Mejora Continua PDCA

PLAN	DO	CHECK	ACT	responsabilidad	fecha de selección
Falta de especificidad local: Los trámites varían por distrito, pero la UI mostraba datos genéricos.	Botón de Municipalidad: Se añadió un selector de municipalidad en la esquina superior derecha del UI.	Al cambiar de distrito, la información de ubicación y requisitos se actualiza.	Implementar un estado global (Context API en React) para manejar la municipalidad activa en toda la app.	Edhu	29/05/2026
Desconocimiento de anexos: Los usuarios no sabían de dónde sacar los formularios (ej. FUT).	Sección Descargas: Se integró el bloque "Formatos para descargar" en el wireframe y UI.	Los usuarios pueden identificar visualmente el icono de descarga del documento PDF.	Estandarizar una tabla en PostgreSQL para vincular URLs de documentos a cada trámite.	Edhu	29/05/2026
Incertidumbre en el flujo: Tener los requisitos no explica el orden de las acciones a realizar.	Timeline de Pasos: Se diseñó la sección "Pasos a seguir" (1. Preparar, 2. Acercarse, 3. Mesa de partes).	La lectura secuencial reduce la confusión del ciudadano.	Mantener un componente React reusable <Stepper /> para los pasos de cualquier trámite.	Edhu	29/05/2026
Falta de información rápida: El usuario tenía dudas comunes, pero la app no tenía sección de FAQ (solo un enlace inactivo), forzándolo a abandonar la página o acudir presencialmente.	Componente FAQ en Modal: Se implementó FaqModal.jsx (acordeón de preguntas) y se activó mediante botones estratégicos en el Footer y el Sidebar.	Al hacer clic en "Preguntas Frecuentes", el ciudadano accede instantáneamente a las respuestas sin salir de la vista actual ni recargar la página.	Estandarizar el uso de Modales superpuestos para información complementaria (FAQ, Términos).	Leonardo	3/06/2026
Problema de ubicación: El usuario preparaba sus papeles pero no sabía a qué edificio ir.	Geolocalización: Se añadió la caja "¿Dónde ir?" vinculada al distrito seleccionado.	Se muestra claramente la dirección física (Ej. Plaza Central de Carabayillo).	Agregar futuras integraciones con Google Maps o Leaflet en este componente.	Jesus	31/05/2026
Lógica de IA mezclada en UI: El componente del Chatbot contenía demasiada lógica de simulación (setTimeout).	Servicio Frontend: Se movió la lógica simulada a un archivo independiente iaService.js.	El componente PanelChatbot.jsx queda limpio y solo maneja renderizado.	Mantener esta separación de responsabilidades; facilitará cambiar el mock por una API real luego.	Leonardo	31/05/2026
Experiencia visual del Chatbot: La IA respondía instantáneamente, sintiéndose poco natural.	Indicador de "Escribiendo": Se añadió un estado de carga UI simulando el tiempo de procesamiento.	La experiencia interactiva es mucho más realista y amigable.	Estandarizar un componente <LoaderChat /> para todas las interacciones futuras con IA.	Leonardo	31/05/2026
Envío de mensajes vacíos: El usuario podía hacer clic en "Enviar" sin escribir nada en el chat.	Validación de Input: Se implementó una restricción en el formulario del chatbot en React.	Ya no se procesan ni renderizan burbujas de texto en blanco.	Aplicar validaciones de campos (formularios) en toda la aplicación de manera estándar.	Edhu	31/05/2026
Arquitectura de Software (Backend): Riesgo de acoplamiento al conectar la BD.	Adaptadores Hexagonales: Se crearon puertos (TramiteRepositoryPort) para aislar la lógica.	El controlador REST no toca la BD directamente, usa el servicio de dominio.	Respetar estrictamente los principios de Arquitectura Hexagonal en los próximos sprints.	Leonardo	2/06/2026
Historia de Usuario (Formato): Las historias previas no seguían el estándar BDD exigido en la guía.	Refinamiento BDD: Se reescribió la HU usando la plantilla Rol-Necesito-Para y Given-When-Then.	La historia ahora cubre las nuevas secciones (descargas, municipalidad, pasos).	Utilizar siempre este estándar para definir los requisitos en las siguientes iteraciones.	Jesus	31/05/2026
Organización de Repositorio: Mezcla de componentes en una sola carpeta en frontend.	Reestructuración de carpetas: Se organizó /components separando /layout, /chatbot y /tramite.	El código es mucho más fácil de navegar y mantener por el equipo.	Documentar esta estructura en el README.md del repositorio oficial.	Jesus	31/05/2026

2.11. Lista de verificación de revisión GitHub

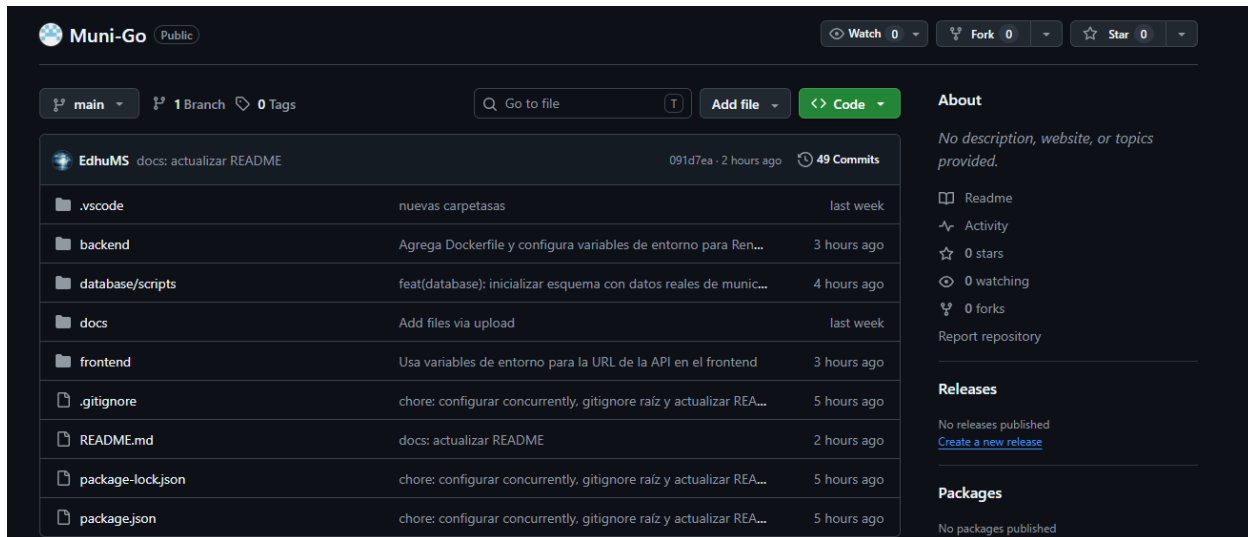
Elemento a revisar	Cumple
Carpeta controllers (Adapters In)	Sí

Carpeta models (Domain)	Sí
Carpeta views (Components React)	Sí
Carpeta database (Infra/PostgreSQL)	Sí
Carpeta integration (iaService / Ports)	Sí
Separación MVC/Hexagonal correcta	Sí
Clase conexión BD	Sí
Clase integración IA (Servicio Simulado)	Sí
README técnico	Sí
Commits organizados	Sí

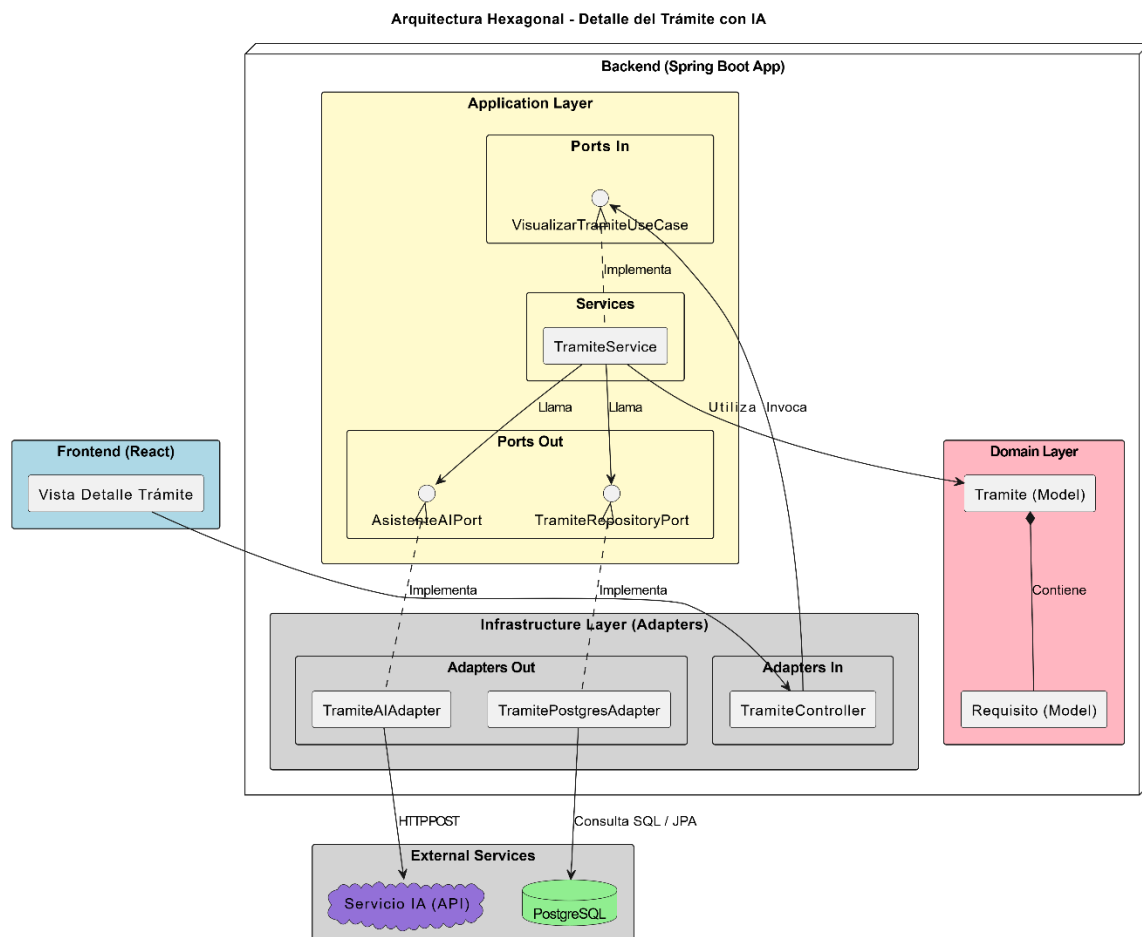
2.12. Evidencias requeridas

- **URL del repositorio:** <https://github.com/Estudiante-leonardo/Muni-Go.git>
- **URL de la web funcional:** <https://muni-go-a6xj.onrender.com/>

- Capturas de GitHub:



- Capturas de arquitectura:



- Capturas del wireframe:

The wireframe shows a web browser window with the URL `https://MuniGo/Descripcion_Catalogo`. The page header includes a menu icon, a 'Volver al Catálogo' link, and the 'MuniVirtual' logo. The main heading is 'Trámite: Licencia de Funcionamiento Comercial'. Below this, it lists 'Costo: S/ 150', 'dias habiles: 5 dias', and 'tipo: Virtual'. A 'Resumen Inteligente IA' box states that this process allows opening smaller businesses (under 100m2) with just a DNI, rental contract, and fire extinguisher. A 'Requisitos Oficiales' section lists three requirements: 'DNI vigente', 'Declaración Jurada', and 'Croquis de Distribución', each with an unchecked checkbox. A 'Formatos para descargar' section provides a link to the 'Formato Único de Trámite (FUT)' (120 KB PDF). A 'Pasos a seguir' section outlines three steps: 1. 'Preparar Expediente' (gather documents and fill forms), 2. 'Acercarse a Sede' (go to the Municipality of Carabayillo), and 3. 'Mesa de Partes' (submit documents and wait for results). A location section '¿Dónde ir?' points to 'Municipalidad de Carabayillo' at 'Plaza Central del distrito' with hours 'L-V de 8:00 AM a 4:30 PM'. On the right, an 'Asistente de IA del Trámite' box explains that the distribution croquis can be hand-drawn for small businesses. At the bottom right, a text input field asks 'Pregúntale a la IA sobre este trámite... Ej: ¿Cómo hago el croquis?' with an 'Enviar' button.

2.13. Reflexión final

La aplicación del ciclo PDCA nos ha permitido darnos cuenta de que la mejora continua no solo se aplica al código backend, sino fuertemente a la Experiencia del Usuario (UX). Al evaluar nuestro primer wireframe, detectamos que brindar solo requisitos no era suficiente. Planificamos e implementamos mejoras tangibles: descarga de formatos, flujos de pasos y geolocalización vinculada a un selector de municipalidades. Aislar la IA en un servicio simulado nos permite tener un MVP funcional y testeable

hoy, preparando el terreno arquitectónico para conectar una API real de Inteligencia Artificial en el siguiente ciclo, asegurando siempre un software escalable y centrado en el ciudadano.